



Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	C01 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192425147
Student Name	D.Hasini

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of “ShopSphere”, an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds $n = 100,000$ products and is expected to reach $n = 1,000,000$ within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n - 2 DO
  FOR j ← i + 1 TO n - 1 DO
    IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
      MarkDuplicate(j)
    END IF
  END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is $\Theta(n \log n)$ because the outer loop runs about $\log n$ times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
  FOR j ← 0 TO i - 1 DO
    Audit(Request[j])           // basic operation
  END FOR
  i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by

comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size $n/2$ (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for $n = 1,024$.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a , b , $f(n)$, $n^{\log_b a}$ and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p , and evaluate it for $n = 1,000,000$ and $p = 0.5$.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log n + 50$ is $\Theta(n^2)$ by finding constants c_1 , c_2 and n_0 , and show that $f(n)$ is not $O(n)$.

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for $n = 1,000,000$. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach

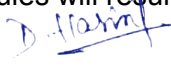
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I D.Hasini / 192425147 (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:  Date: 10-10-26

Solution: Scalability Audit of “ShopSphere”

CSA06 – Design and Analysis of Algorithms | CO1 – Assessment Tool 1

Student Name: D.Hasini Reg. No.: 192425147

Conventions and assumptions used in every part

- $\log$ means $\log_2$; $n = 1,000,000$ is the expected catalogue size within a year ($n = 100,000$ today).
- Server speed: 10^9 basic operations per second (the exponent of this figure is not legible in the question sheet, so 10^9 is assumed; a 10^8 check is given in part (g)). Nightly window = 10 min = 600 s, i.e. a budget of 6×10^{11} operations.
- Recurrences are solved for $n = 2^k$ with $T(1) = 1$; the results extend to all n by the usual floor/ceiling argument.
- Every count and table in this report was produced by the Python program **scalability_audit.py**; the screenshots show my name, registration number and the system date/time printed by the program itself.

(a) Understand the Problem

The table gives, for each module, the input size, the basic operation (the statement executed most often, whose count decides the running time), where it occurs and the growth expected as n grows.

Module	Input size	Basic operation and where it occurs	Expected growth (justification)
M1 Duplicate-Listing Detector	n = number of products	The title/seller comparison in the IF statement of the inner loop; it runs once for every pair (i, j) with $i < j$.	Quadratic, about $n^2/2$. Every pair is compared with no early exit, so ten times more products means about 100 times more work.
M2 API Rate Limiter	n = limit on the window size (requests)	Audit(Request[j]) in the inner loop; the window i takes the values 1, 2, 4, ... below n .	Linear, between n and $2n$. The window sizes form a geometric series $(1 + 2 + 4 + \dots)$, dominated by its last term.
M3 – S1 sequential search	n = products in the unsorted catalogue	Comparison of the key with Product[i] inside the scan loop.	Linear: up to n comparisons per search (about $n/2$ on average when the product exists).
M3 – S2 binary search	n = entries in the sorted index	Comparison of the key with the middle entry; each comparison discards half the index.	Logarithmic, about $\log n + 1$. Ten times more data adds only about 3.3 comparisons.
M4 Recommendation Engine	n = size of the customer-product data	The linear-time merge of partial results (about n steps) in every recursive call.	Current design $T(n) = 4T(n/2) + n$ grows like n^2 ; the redesign $T(n) = 2T(n/2) + n$ grows like $n \log n$.
M5 Sales Analytics Engine	n = number of sales records	The sort by revenue done at every level: about $m \log m$ comparisons for a subproblem of size m .	$T(n) = 2T(n/2) + n \log n$ grows like $n \log^2 n$: faster than $n \log n$, much slower than n^2 .
Report generator f(n)	n = records reported	Every elementary operation counted by $f(n) = 3n^2 + 10n \log n + 50$.	The $3n^2$ term dominates for large n , so the cost is quadratic.

(b) Analyse the Iterative Modules

M1 – Duplicate-Listing Detector

For a fixed i the inner loop runs $j = i + 1, \dots, n - 1$, that is $(n - 1) - (i + 1) + 1 = n - 1 - i$ times. Summing over $i = 0, \dots, n - 2$:

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} (n - 1 - i) = (n - 1) + (n - 2) + \dots + 2 + 1 \\ &= (n - 1) \cdot n / 2 = \mathbf{n(n - 1)/2 = n^2/2 - n/2} \text{ comparisons} \end{aligned}$$

The count does not depend on the data (the loop never exits early), so best, average and worst case are all the same: **time $\Theta(n^2)$** . Only the variables i and j are used besides the input, so **space $\Theta(1)$** . Values: $n = 100,000 \rightarrow 4,999,950,000$ comparisons; $n = 1,000,000 \rightarrow 499,999,500,000 \approx 5 \times 10^{11}$. Meaning: when the catalogue grows tenfold, the nightly duplicate scan takes about a hundred times longer.

M2 - API Rate Limiter

The outer loop runs while $i < n$ with $i = 2^k$ ($k = 0, 1, 2, \dots$), so it runs for $k = 0, \dots, \lceil \log n \rceil - 1$, i.e. $\lceil \log n \rceil$ times. In the round with $i = 2^k$ the inner loop executes Audit exactly $i = 2^k$ times (not n times):

$$A(n) = \sum_{k=0}^{\lceil \log n \rceil - 1} 2^k = 2^{\lceil \log n \rceil} - 1 \quad (\text{geometric series})$$

Because $n \leq 2^{\lceil \log n \rceil} < 2n$, $n - 1 \leq A(n) \leq 2n - 2$, and for $n = 2^m$ exactly $A(n) = n - 1$.

The claim is incorrect. The argument “log n rounds $\times$ up to n operations” only gives the upper bound $O(n \log n)$; it is not tight because the inner loop does not run n times in every round. Round k costs only 2^k ; the rounds cost 1, 2, 4, ..., $n/2$ and add up to about n . **The tight bound is $\Theta(n)$** (time) and $\Theta(1)$ extra space. For $n = 10^6$ that is 1,048,575 operations instead of the 2×10^7 that $n \log n$ would suggest.

Verification for $n = 1,024$: the loop runs for $i = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512$ (10 rounds; $i = 1024$ is not $< n$), so

$$A(1024) = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 2^{10} - 1 = 1023 = n - 1$$

The claimed $n \log n$ would give $1024 \times 10 = 10,240$, ten times too large; the program below measures 1023 Audit calls.

```
Terminal - python3 scalability_audit.py b

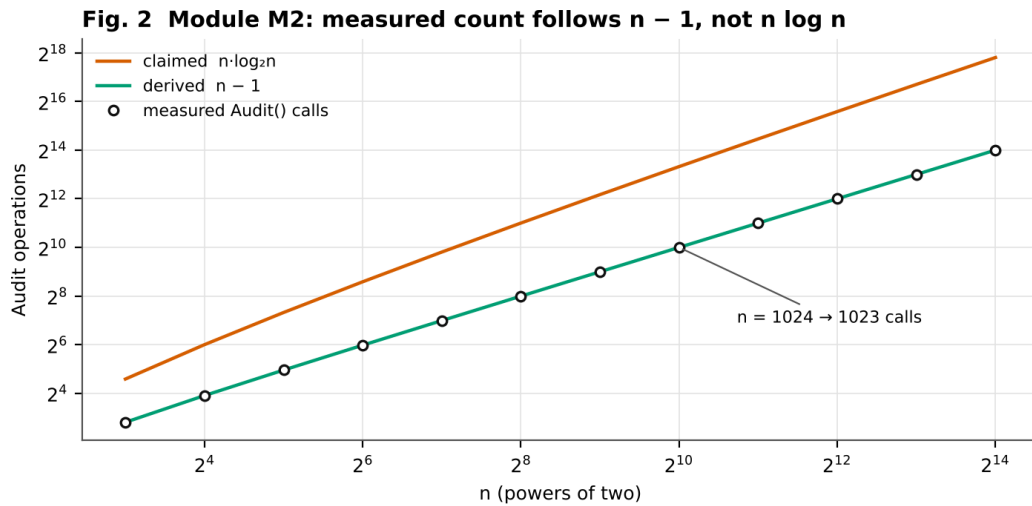
$ python3 scalability_audit.py b
=====
Part (b) Iterative modules M1 and M2
Name: D.Hasini   Reg. No.: 192425147
System date/time: 10-10-2026 14:14:20
=====

M1 comparisons (measured vs n(n-1)/2)
  n    measured    n(n-1)/2  match
  10         45         45    True
  100       4950       4950    True
  500      124750     124750    True
 1000     499500     499500    True
 2000    1999000    1999000    True
formula n=100,000: 4,999,950,000 comparisons
formula n=1,000,000: 499,999,500,000 comparisons

M2 Audit operations (measured vs 2^ceil(log2 n) - 1)
  n    measured    formula    n*log2(n)
  16         15         15         64
  100        127        127        664
 1000       1023       1023       9966
 1024       1023       1023      10240
 4096       4095       4095      49152
100000     131071     131071     1660964

Verification n = 1024: window sizes i = [1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
sum = 1023 = n - 1 = 1023   measured = 1023
claimed n*log2(n) = 10240 -> claim is NOT correct, M2 is Theta(n)
```

Screenshot 1 - output of scalability_audit.py b (M1 and M2 counts, verification for $n = 1024$).



(c) Solve a Recurrence by Substitution (search S2)

Recurrence: $T(n) = T(n/2) + 1$, $T(1) = 1$. Backward substitution:

$$\begin{aligned}
 T(n) &= T(n/2) + 1 \\
 &= [T(n/4) + 1] + 1 = T(n/4) + 2 \\
 &= [T(n/8) + 1] + 2 = T(n/8) + 3 \\
 &\vdots \\
 &= T(n/2^k) + k \quad (\text{general step after } k \text{ substitutions})
 \end{aligned}$$

The recursion stops when $n/2^k = 1$, i.e. $k = \log_2 n$, giving

$$T(n) = T(1) + \log_2 n = \mathbf{1 + \log_2 n} \quad (\text{for general } n: \lfloor \log_2 n \rfloor + 1)$$

Check by induction: $T(n) = T(n/2) + 1 = (1 + \log n - 1) + 1 = 1 + \log n$. Examples: $T(1) = 1$, $T(8) = 4$, $T(1024) = 11$, and for $n = 1,000,000$ the exact count is $\lfloor \log_2 10^6 \rfloor + 1 = 20$ comparisons.

Implementation	Time complexity	Space complexity	Reason
Iterative binary search	$\Theta(\log n)$	$\Theta(1)$	A loop with lo, hi, mid only; at most $\lfloor \log n \rfloor + 1$ iterations.
Recursive binary search	$\Theta(\log n)$	$\Theta(\log n)$	Same comparisons, but each call adds a stack frame; depth is $\lfloor \log n \rfloor + 1$ (20 frames for $n = 10^6$).

Both versions are logarithmic in time; the iterative one is preferable on a server because it needs no stack.

```

Terminal - python3 scalability_audit.py c

$ python3 scalability_audit.py c
=====
Part (c) Recurrence of search S2:  $T(n) = T(n/2) + 1$ ,  $T(1) = 1$ 
Name: D.Hasini Reg. No.: 192425147
System date/time: 10-10-2026 14:14:20
=====

n  T(n) recursion  1 + log2 n
1      1          1
2      2          2
4      3          3
8      4          4
32     6          6
1024   11         11
32768  16         16
1048576 21        21

n = 1,000,000: floor(log2 n) + 1 = 20
measured worst-case comparisons on sample keys = 20
Time: Theta(log n) | Space: iterative Theta(1), recursive Theta(log n)

```

Screenshot 2 - output of scalability_audit.py c ($T(n)$ from the recurrence equals $1 + \log_2 n$).

(d) Apply the Master Theorem

For $T(n) = aT(n/b) + f(n)$: compare $f(n)$ with $n^{\log_b a}$. Case 1: $f = O(n^{\log_b a - \epsilon}) \Rightarrow \Theta(n^{\log_b a})$; Case 2: $f = \Theta(n^{\log_b a}) \Rightarrow \Theta(n^{\log_b a} \log n)$; Case 3: $f = \Omega(n^{\log_b a + \epsilon})$ with the regularity condition $\Rightarrow \Theta(f(n))$.

M4 - Recommendation Engine

	a	b	f(n)	$n^{\log_b a}$	Case	Result
Current: $T(n) = 4T(n/2) + n$	4	2	n	$n^{\log_2 4} = n^2$	1: $f(n) = n = O(n^{2-\epsilon})$ with $\epsilon = 1$	$\Theta(n^2)$ (exact $2n^2 - n$)
Redesign: $T(n) = 2T(n/2) + n$	2	2	n	$n^{\log_2 2} = n$	2: $f(n) = \Theta(n)$	$\Theta(n \log n)$ (exact $n \log n + n$)

When the data doubles. Current design: $T(2n)/T(n) = (8n^2 - 2n)/(2n^2 - n) \rightarrow 4$, so doubling the data quadruples the running time ($2 \times 10^{12} \rightarrow 8 \times 10^{12}$ operations for $n = 10^6 \rightarrow 2 \times 10^6$). Redesign: $T(2n)/T(n) = 2(1 + 1/(\log n + 1)) \approx 2.1$ (2.10 at $n = 2^{19}$), i.e. slightly more than double, almost linear growth. Removing the replicated partitions turns the dominant term from n^2 into $n \log n$.

M5 - Sales Analytics Engine: $T(n) = 2T(n/2) + n \log n$

Here $a = 2$, $b = 2$, $f(n) = n \log n$ and $n^{\log_2 2} = n$. **Case 1** needs f smaller than n : no. **Case 2** needs $f = \Theta(n)$: no, because $f(n)/n = \log n \rightarrow \infty$. **Case 3** needs $f = \Omega(n^{1+\epsilon})$ for some $\epsilon > 0$: no, because $n \log n / n^{1+\epsilon} = \log n / n^\epsilon \rightarrow 0$ for every $\epsilon > 0$. So **none of the three standard cases applies** ($f(n)$ lies in the gap between n and $n^{1+\epsilon}$).

Extended Master Theorem: if $f(n) = \Theta(n^{\log_b a} \log^k n)$ with $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$. With $k = 1$: **$T(n) = \Theta(n \log^2 n)$** .

Recursion-tree confirmation ($n = 2^L$, $L = \log n$):

Level i	Nodes	Size of each node	Cost per node	Cost of the level
0	1	n	$n \log n$	$n \cdot L$
1	2	$n/2$	$(n/2) \log(n/2)$	$n \cdot (L - 1)$
2	4	$n/4$	$(n/4) \log(n/4)$	$n \cdot (L - 2)$
...	...	...	...	...
$L - 1$	$n/2$	2	$2 \cdot \log 2 = 2$	$n \cdot 1$
L (leaves)	n	1	$T(1) = 1$	n

$$T(n) = \sum_{i=0}^{L-1} n(L-i) + n = n \cdot [L + (L-1) + \dots + 1] + n = \mathbf{n \cdot L(L+1)/2 + n = \Theta(n \log^2 n)}$$
 For $n = 10^6$ ($L \approx 19.93$): $T \approx 2.1 \times 10^8$ operations.

Meaning: M5 is a little worse than an $n \log n$ algorithm but fully acceptable at $n = 10^6$; the ratio $T(n)/(n \log^2 n)$ in the output settles near $\frac{1}{2}$, as the formula predicts.

```

Terminal - python3 scalability_audit.py d

$ python3 scalability_audit.py d
=====
Part (d) Master Theorem recurrences M4 and M5 (n = 2^k, T(1) = 1)
Name: D.Hasini Reg. No.: 192425147
System date/time: 10-10-2026 14:14:20
=====

      n      4T(n/2)+n      2n^2-n      2T(n/2)+n      n log n+n
      2           6           6           4           4
      16          496          496          80          80
      256        130816        130816        2304        2304
      4096       33550336       33550336       53248       53248
      65536     8589869056     8589869056     1114112     1114112
      1048576  2199022206976  2199022206976  22020096  22020096

Doubling the data (n -> 2n):
n=2^10 current x4.001 redesign x2.182
n=2^15 current x4.000 redesign x2.125
n=2^19 current x4.000 redesign x2.100

M5: T(n) = 2T(n/2) + n log n (recursion tree: n*L(L+1)/2 + n, L = log2 n)
      n      T(n)      n*L(L+1)/2+n      T/(n log2^2 n)
      16          176          176          0.688
      256         9472         9472          0.578
      4096        323584        323584          0.549
      65536       8978432       8978432          0.535
      1048576    221249536     221249536          0.527
  
```

Screenshot 3 - output of scalability_audit.py d (recurrences evaluated directly match the closed forms).

(e) Analyse Best, Average and Worst Cases

Search	Best case	Average case	Worst case
S1 sequential (unsorted catalogue)	The product is the first record: 1 comparison.	Present with equal probability at every position: $(1 + 2 + \dots + n)/n = \mathbf{(n + 1)/2}$.	The product is the last record or is absent: n comparisons.
S2 binary (sorted index)	The key is the middle entry: 1 comparison.	Successful search $\approx \log_2 n - 1$ (half of all entries lie at the deepest level of the comparison tree).	The key is found at the last level or is absent: $\lceil \log_2 n \rceil + 1$ comparisons.
For $n = 1,000,000$	S1: 1 S2: 1	S1: 500,000.5 (present) S2: ≈ 18.9	S1: 1,000,000 S2: 20

Average number of comparisons for S1 when the product exists with probability p

With probability p the key is present and (uniformly likely at any position) costs $(n + 1)/2$ comparisons; with probability $1 - p$ it is absent and costs n :

$$A(n, p) = p \cdot (n + 1)/2 + (1 - p) \cdot n = \mathbf{n(1 - p/2) + p/2}$$

$n = 1,000,000$ and $p = 0.5$: $A = 0.5 \times 500,000.5 + 0.5 \times 1,000,000 = 250,000.25 + 500,000 = \mathbf{750,000.25 \approx 7.5 \times 10^5 \text{ comparisons}}$

Meaning: an average S1 query examines three-quarters of a million records, whereas the worst S2 query needs only 20. The program reproduces the formula for $n = 1,000$ (750.25) and measures the S2 values on $n = 1,000,000$.

```

Terminal - python3 scalability_audit.py e

$ python3 scalability_audit.py e
=====
Part (e) Best / average / worst case of S1 and S2
Name: D.Hasini Reg. No.: 192425147
System date/time: 10-10-2026 14:14:20
=====
S1 (n=1000): best=1 worst(found last)=1000 not found=1000
S1 average, key present = 500.5 (formula (n+1)/2 = 500.5)
S1 average with p=0.5: measured 750.25 formula 750.25
S1 average n=1,000,000, p=0.5 : 750,000.25 comparisons

S2 (n=1,000,000): best=1 worst=20 (= floor(log2 n)+1 = 20)
S2 average over 142,858 sampled present keys = 18.95 (~ log2 n - 1 = 18.93)

```

Screenshot 4 - output of scalability_audit.py e (best/average/worst comparisons of S1 and S2).

(f) Prove an Asymptotic Bound

Definition: $f(n) = \Theta(g(n))$ if there are constants $c_1, c_2 > 0$ and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$. Here $f(n) = 3n^2 + 10n \log n + 50$ and $g(n) = n^2$.

Lower bound. For $n \geq 1$ the terms $10n \log n$ and 50 are non-negative, so $f(n) \geq 3n^2$. Take $c_1 = 3$.

Upper bound. For $n \geq 1$ we have $\log n \leq n$, hence $10n \log n \leq 10n^2$, and $50 \leq 50n^2$. Therefore

$$f(n) = 3n^2 + 10n \log n + 50 \leq 3n^2 + 10n^2 + 50n^2 = 63n^2 \quad \text{for all } n \geq 1. \text{ Take } c_2 = 63.$$

Constants: $c_1 = 3, c_2 = 63, n_0 = 1 \Rightarrow 3n^2 \leq f(n) \leq 63n^2$ for all $n \geq 1$, so $f(n) = \Theta(n^2)$.

(A tighter choice is $c_2 = 4$ with $n_0 = 60$: from $n = 60$ on, $10n \log n + 50 \leq n^2$; for example $n = 64$ gives $3,890 \leq 4,096$.)

$f(n)$ is not $O(n)$. Suppose $f(n) \leq c \cdot n$ for all $n \geq n_0$ for some constants c, n_0 . Since $f(n) \geq 3n^2$, this gives $3n^2 \leq c \cdot n$, i.e. $n \leq c/3$, for every $n \geq n_0$. But for $n = \max(n_0, \lfloor c/3 \rfloor + 1)$ this is false — a contradiction. Equivalently $f(n)/n \geq 3n \rightarrow \infty$. Hence $f(n) = \Omega(n^2)$ grows strictly faster than n .

Meaning: the nightly report generator is quadratic. At $n = 10^6$ it performs about 3×10^{12} operations (3,000 s at 10^9 operations per second), which is five times the 10-minute window, so it needs the same attention as M4.

```

Terminal - python3 scalability_audit.py f

$ python3 scalability_audit.py f
=====
Part (f) f(n) = 3n^2 + 10 n log2 n + 50 is Theta(n^2)
Name: D.Hasini Reg. No.: 192425147
System date/time: 10-10-2026 14:14:21
=====
c1=3, c2=63, n0=1: c1*n^2 <= f(n) <= c2*n^2 for 1<=n<=200000 -> True
tighter c2=4: holds for n0=60 <= n <= 200000 -> True (last failure at n=59)

      n          f(n)  f(n)/n^2      f(n)/n
    ----
      10             682    6.8219      68.2
     100          36,694   3.6694     366.9
    1000       3,099,708   3.0997    3,099.7
   10000     301,328,821   3.0133   30,132.9
  100000  30,016,609,690   3.0017  300,166.1
 1000000 3,000,199,315,736   3.0002 3,000,199.3
f(n)/n grows without bound -> f(n) is not O(n)

```

Screenshot 5 - output of scalability_audit.py f (the bounds hold; $f(n)/n^2 \rightarrow 3$ and $f(n)/n$ grows without bound).

(g) Scalability Decision

Speed 10^9 operations/s, $n = 1,000,000$, window 600 s (6×10^{11} operations).

Module	Complexity	Operations at $n = 10^6$	Running time	10-minute window
M1	$\Theta(n^2)$	$n(n-1)/2 = 499,999,500,000 \approx 5.0 \times 10^{11}$	500 s (8.3 min)	Meets, but only 17% spare
M2	$\Theta(n)$	$2^{20} - 1 = 1,048,575 \approx 1.05 \times 10^6$	0.001 s	Meets
S2 (per search)	$\Theta(\log n)$	$\lfloor \log n \rfloor + 1 = 20$	2×10^{-8} s	Meets
M4 current	$\Theta(n^2)$	$2n^2 - n = 1,999,999,000,000 \approx 2.0 \times 10^{12}$	2,000 s (33.3 min)	FAILS
M4 redesign	$\Theta(n \log n)$	$n \log n + n = 20,931,569 \approx 2.1 \times 10^7$	0.021 s	Meets
M5	$\Theta(n \log^2 n)$	$n \cdot L(L+1)/2 + n = 209,599,497 \approx 2.1 \times 10^8$	0.21 s	Meets

```

Terminal - python3 scalability_audit.py g

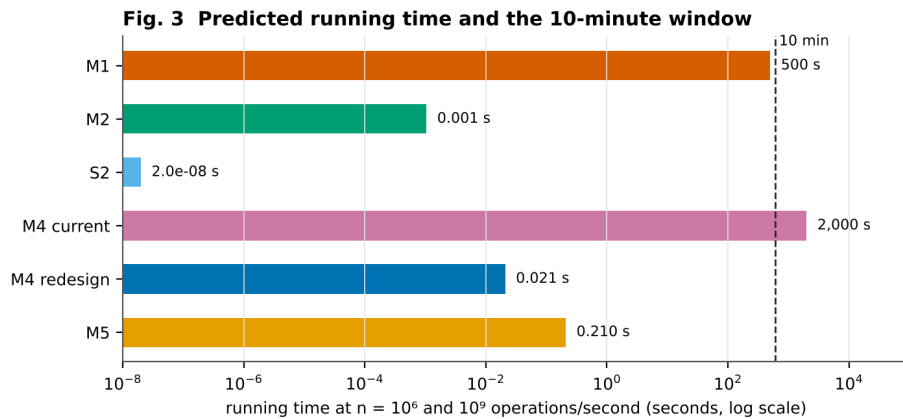
$ python3 scalability_audit.py g
=====
Part (g) Scalability at n = 1,000,000
Name: D.Hasini   Reg. No.: 192425147
System date/time: 10-10-2026 14:14:21
=====

speed = 1e+09 ops/s, window = 600s (budget 6.0e+11 ops)
module      operations      time (s)  verdict
M1 n(n-1)/2      499,999,500,000  499.999500  MEETS
M2 2^ceil(lg n)-1    1,048,575      0.001049  MEETS
S2 floor(lg n)+1      20            2.0e-08  MEETS
M4 current 2n^2-n    1,999,999,000,000  1999.999000  FAILS
M4 redesign n lg n+n  20,931,569     0.020932  MEETS
M5 n L(L+1)/2+n    209,599,497    0.209599  MEETS

M1 stays inside the window only up to n = 1,095,445
Sensitivity at 1e8 ops/s: M1 = 5,000s, M4 current = 20,000s -> both FAIL

```

Screenshot 6 – output of scalability_audit.py g (operation counts, times and verdicts at $n = 1,000,000$).



Decision

Modules that fail the window: M4 in its current form (2,000 s against 600 s; it would need about 3.3 times the window). **M1** still fits at $n = 10^6$ (500 s), but only with a 100-second margin: it stays inside the window only up to $n \approx 1,095,445$, i.e. about 9.5% above the forecast, and it shares the night with the other jobs. At a speed of 10^8 operations/s, M1 (5,000 s) and M4 current (20,000 s) would both fail clearly.

Modules that must be redesigned:

Module	Action	Justification
M4 (mandatory)	Adopt the redesign without replicated partitions: $T(n) = 2T(n/2) + n$.	$\Theta(n^2) \rightarrow \Theta(n \log n)$: $2 \times 10^{12} \rightarrow 2.1 \times 10^7$ operations (about 95,000 times fewer), 2,000 s $\rightarrow$ 0.02 s; doubling the data now costs about 2.1 $\times$ instead of 4 $\times$.

Module	Action	Justification
M1 (strongly recommended)	Replace the pairwise scan by a hash set keyed on (title, seller): one pass, report an item as duplicate when its key is already in the set. Alternative: sort by (title, seller) and compare neighbours.	$\Theta(n^2) \rightarrow \Theta(n)$ expected (about 10^6 operations, ≈ 1 ms) or $\Theta(n \log n)$ deterministic ($\approx 2 \times 10^7$). The price is $\Theta(n)$ extra memory, a good trade for about 500,000 times fewer operations.
Report generator (advise)	Reduce the n^2 term of $f(n)$ (for example by aggregating with a hash table or a sort).	3×10^{12} operations $\approx 3,000$ s at $n = 10^6$ exceeds the window.

Keep as they are: M2 (already linear, 1 ms; the $\Theta(n \log n)$ claim was wrong and no redesign is needed), S2 for product search (20 comparisons per query against about 7.5×10^5 on average for S1, so S2 must be the production version; sorting the index once costs $\Theta(n \log n)$), and M5 (0.21 s; if it ever becomes a bottleneck, sort once and merge at each level, which gives $T(n) = 2T(n/2) + n = \Theta(n \log n)$).